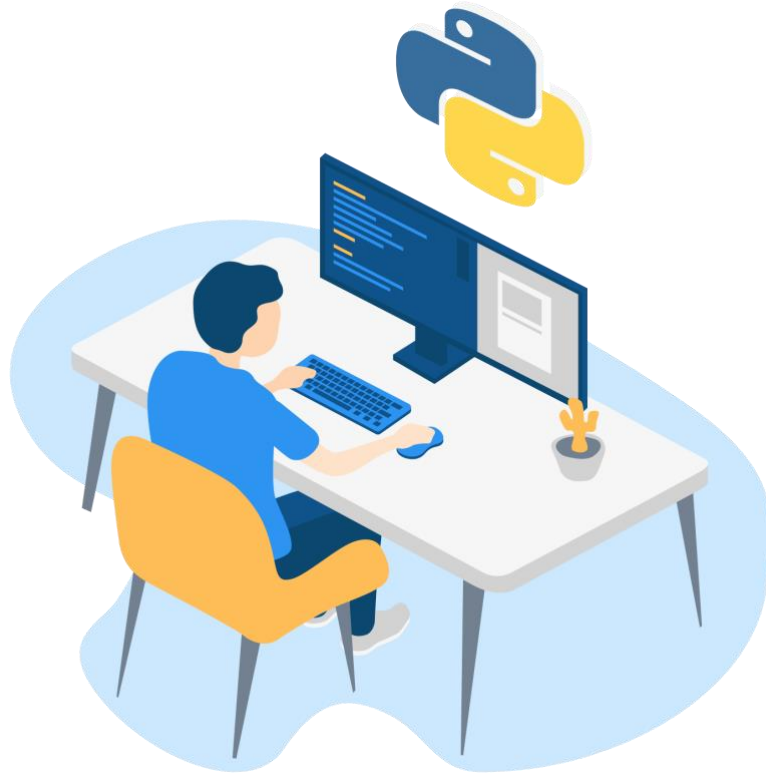


BAB 7

DJANGO URL, Views dan Template



7.1 Tujuan

Setelah menyelesaikan bab ini dan mengikuti kegiatan praktikum, mahasiswa diharapkan mampu:

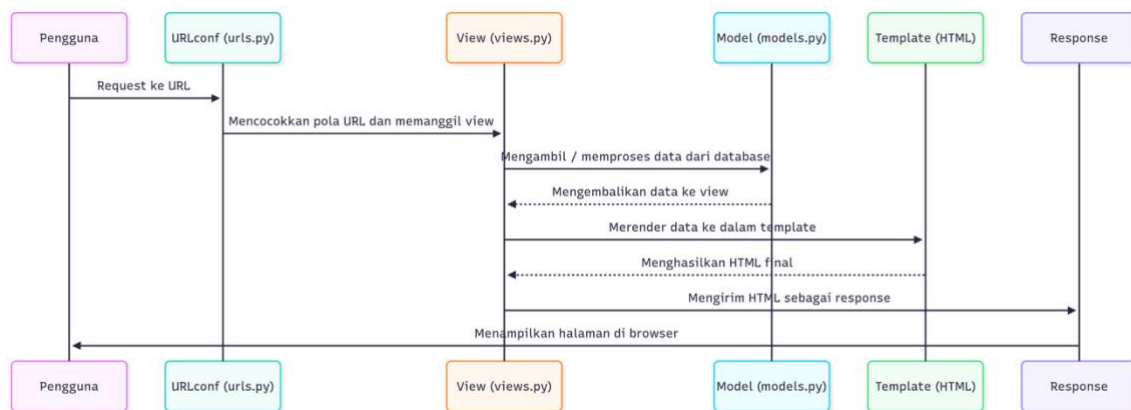
1. Menjelaskan alur kerja fundamental Django yang menghubungkan URL (*Uniform Resource Locator*), *View*, dan *Template*.
2. Membuat dan mengonfigurasi pola URL (*URL patterns*) untuk memetakan alamat web ke logika bisnis.
3. Mengembangkan *views* fungsional untuk mengambil data dari model basis data dan mempersiapkannya untuk ditampilkan.
4. Merender data dinamis pada *template* HTML menggunakan Django Template Language (DTL)

7.2 Pengantar

Pada Bab 6, kita telah berhasil membangun fondasi data aplikasi blog kita melalui Django Models dan mengelolanya via panel admin. Namun, data tersebut belum bisa dilihat oleh pengguna umum di situs web kita. Bab ini akan membahas "jantung" dari arsitektur Django, yaitu bagaimana sebuah permintaan (*request*) dari pengguna diproses dan diubah menjadi halaman web yang bisa dilihat.

7.2.1 Alur Kerja Permintaan-Respons di Django

Ketika seorang pengguna mengunjungi sebuah URL di situs Django, serangkaian proses terjadi di belakang layar.



1. URL (URLconf): Pengguna mengakses sebuah alamat web. Django menerima alamat ini dan mencocokkannya dengan daftar pola URL yang telah kita definisikan di file `urls.py`. Analoginya seperti seorang resepsionis yang mengarahkan tamu ke ruangan yang tepat.
2. View: Setelah pola URL yang cocok ditemukan, Django akan memanggil fungsi *View* yang terkait. *View* adalah fungsi Python yang berisi semua logika bisnis. Tugasnya adalah memproses permintaan, berinteraksi dengan basis data (melalui Model), dan memutuskan data apa yang perlu ditampilkan.
3. Template: *View* tidak menghasilkan halaman HTML secara langsung. Sebaliknya, ia akan mengambil sebuah file *Template* (file HTML dengan sintaks khusus) dan menyisipkan data yang telah disiapkannya ke dalam *template* tersebut. Proses ini disebut *rendering*.
4. Response: Hasil *rendering* adalah sebuah dokumen HTML lengkap yang kemudian dikirimkan kembali oleh Django ke peramban pengguna untuk ditampilkan.

7.2.2 Django URLs

Konfigurasi URL di Django (sering disebut URLconf) adalah jembatan antara URL yang dilihat pengguna dan kode *view* kita.

- `mysite/urls.py` (Project URLs): File ini adalah file URLconf utama. Biasanya, file ini tidak menangani semua URL secara langsung, melainkan mendelegasikannya ke file `urls.py` milik masing-masing aplikasi menggunakan fungsi `include()`.
- `blog/urls.py` (App URLs): Praktik terbaik adalah setiap aplikasi memiliki file `urls.py`-nya sendiri. Hal ini akan membuat aplikasi menjadi modular dan mudah digunakan kembali di proyek lain.
- Fungsi `path()`: Digunakan untuk mendefinisikan satu rute URL.
- `name='post_list'`: Memberi nama unik pada URL ini sangat penting. Nama ini dapat kita gunakan untuk merujuk ke URL ini di bagian lain dari aplikasi (misalnya di *template*) tanpa harus menuliskan URL-nya secara manual.

```
1. # Contoh di blog/urls.py
2. from django.urls import path
3. from . import views
4.
5. urlpatterns = [
6.     path('', views.post_list, name='post_list'),
7.     # path('rute-url/', fungsi_view_yang_dipanggil, nama_unik_untuk_url)
8. ]
```

7.2.3 Django Views

Sebuah *view* adalah otak di balik halaman web. Ia menerima objek `HttpRequest` sebagai argumen pertamanya dan harus mengembalikan objek `HttpResponse`.

```
1. # Contoh di blog/views.py
2. from django.shortcuts import render
3. from .models import Post
4.
5. def post_list(request):
6.     # 1. Logic: Mengambil semua objek Post dari database
7.     posts = Post.objects.all().order_by('created_date')
8.
9.     # 2. Render: Mengirim data 'posts' ke template
10.    # dan mengembalikan hasilnya sebagai HttpResponse
11.    return render(request, 'blog/post_list.html', {'posts': posts})
```

- **QuerySet**: `Post.objects.all()` adalah contoh penggunaan ORM Django. Perintah ini mengambil semua data dari tabel `Post` dan mengembalikannya dalam bentuk sebuah *QuerySet*, yang mirip dengan daftar objek Python.

- `render()`: Fungsi *render* mengambil *request*, nama file *template*, dan sebuah *dictionary* (disebut *context*) yang berisi data untuk dikirim ke *template*.

7.2.4 Django Templates

Template di Django adalah file HTML yang disisipi dengan sintaks khusus dari Django Template Language (DTL). DTL memungkinkan kita untuk menampilkan data dinamis dan menggunakan logika pemrograman sederhana langsung di dalam HTML.

- Lokasi Template: Praktik terbaik adalah membuat direktori `templates` di dalam setiap direktori aplikasi (misalnya, `blog/templates/`). Di dalamnya, buat lagi direktori dengan nama yang sama dengan aplikasi (misalnya, `blog/templates/blog/`). Jadi, path lengkapnya adalah `blog/templates/blog/post_list.html`.
- Variabel: Ditulis di antara kurung kurawal ganda: `{{ nama_variabel }}`. Django akan menggantinya dengan nilai dari variabel tersebut. Contoh: `{{ post.title }}`.
- Tags: Ditulis di antara `{%` dan `%}`. Digunakan untuk logika seperti perulangan atau percabangan.
- `{% for ... %}` dan `{% endfor %}` digunakan untuk melakukan iterasi pada sebuah daftar.
- `|linebreaksbr`: merupakan contoh *template filter*, yang digunakan untuk memodifikasi tampilan variabel. Filter ini mengubah jeda baris baru menjadi tag `<br>`.

```

1. {% for post in posts %}
2.     <div>
3.         <h2>{{ post.title }}</h2>
4.         <p>{{ post.text|linebreaksbr }}</p>
5.     </div>
6. {% endfor %}

```

7.3 Kegiatan Praktikum

Langkah 1: Persiapan

1. Buka kembali folder django hasil praktikum pada Bab sebelumnya
2. Aktifkan *virtual environment* pada folder tersebut tersebut.

Terminal

```
myenv\Scripts\activate.bat
```

Langkah 2 : Membuat URL untuk daftar Posting

1. Di dalam direktori aplikasi blog, buat sebuah file baru bernama `urls.py`.
2. Isi file `blog/urls.py` dengan kode berikut untuk mendefinisikan URL utama aplikasi blog.

`blog/urls.py`

```
1. # blog/urls.py
2. from django.urls import path
3. from . import views
4.
5. urlpatterns = [
6.     path('', views.post_list, name='post_list'),
7. ]
```

3. Sekarang, kita perlu "menghubungkan" file `blog/urls.py` ini ke proyek utama. Buka file `mysite/urls.py`.
4. Modifikasi `mysite/urls.py` untuk menggunakan `include()` agar semua URL dari aplikasi blog terdaftar di proyek. Ini berarti setiap permintaan ke URL root (`http://127.0.0.1:8000/`) akan diteruskan ke `blog/urls.py` untuk diproses lebih lanjut.

`mysite/urls.py`

```
1. # mysite/urls.py
2. from django.contrib import admin
3. from django.urls import path, include # Pastikan include di-import
4.
5. urlpatterns = [
6.     path('admin/', admin.site.urls),
7.     path('', include('blog.urls')), # Tambahkan baris ini
8. ]
```

Langkah 3: Membuat View `post_list`

1. Buka file `blog/views.py`.
2. Tulis fungsi `post_list` yang akan mengambil semua data postingan dan merendernya ke sebuah *template*.

blog/views.py

```
1. # blog/views.py
2. from django.shortcuts import render
3. from django.utils import timezone
4. from .models import Post
5.
6. def post_list(request):
7.     # Mengambil semua post yang sudah di-
       publish, diurutkan berdasarkan tanggal publish
8.     posts = Post.objects.filter(published_date__lte=timezone.now()).order_
       by('published_date')
9.     return render(request, 'blog/post_list.html', {'posts': posts})
```

Langkah 4: Membuat Template HTML

1. Di dalam direktori `blog`, buat struktur direktori seperti berikut: `templates/blog`.

```
blog/
├── templates/
│   └── blog/
```

2. Di dalam direktori `blog/templates/blog/`, buat file baru bernama `post_list.html`.
3. Isi file `post_list.html` dengan kode berikut. File ini adalah kerangka HTML yang akan menampilkan data dari *view*.

post_list.html

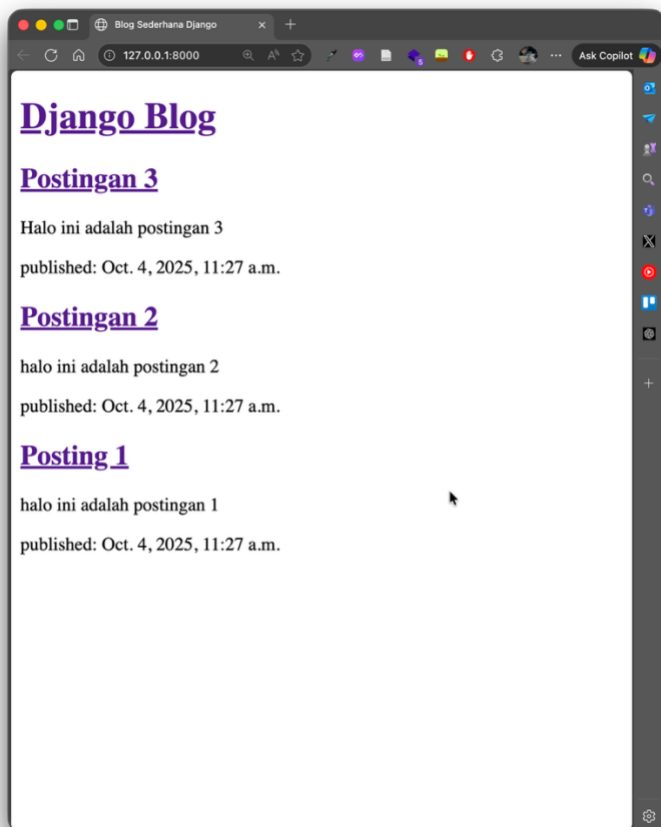
```
1. <!DOCTYPE html>
2. <html>
3. <head>
4.     <title>Blog Sederhana Django</title>
5. </head>
6. <body>
7.     <header>
8.         <h1><a href="/">Blog Saya</a></h1>
9.     </header>
10.
11.     {% for post in posts %}
12.         <article>
13.             <h2><a href="">{{ post.title }}</a></h2>
14.             <p>Published: {{ post.published_date }}</p>
15.             <p>{{ post.text|linebreaksbr }}</p>
16.         </article>
17.     {% endfor %}
18.
19. </body>
20. </html>
```

Jalankan server dan buka browser pada alamat <http://127.0.0.1:8000/>.

Terminal

1. `python manage.py runserver`

Halaman homepage website kita akan menampilkan daftar postingan yang sudah kita buat di Bab 6. Jika tampilan masih kosong maka ulangi proses menambah postingan yang ada pada Bab 6.



7.4 Rangkuman

- Alur kerja Django mengikuti pola URL -> View -> Template. URL memetakan permintaan ke *View*, *View* berisi logika dan mengambil data, dan *Template* bertanggung jawab untuk presentasi.
- Konfigurasi URL yang modular, dengan setiap aplikasi memiliki `urls.py`-nya sendiri dan di-include oleh proyek utama, adalah praktik terbaik.
- *Views* menggunakan Django ORM untuk berinteraksi dengan basis data secara efisien tanpa perlu menulis SQL.
- Django Template Language (DTL) memungkinkan kita menyisipkan data dinamis (`{{ variabel }}`) dan logika (`{% tag %}`) ke dalam file HTML untuk membangun halaman yang dinamis.

7.5 Tugas

Kerjakan tugas-tugas di bawah ini untuk memperkuat pemahaman dan keterampilan Anda.

1. Jelaskan peran dari *context dictionary* (argumen ketiga) dalam fungsi `render()` di sebuah *view*. Apa isinya dan bagaimana ia digunakan di *template*?
2. Mengapa memberikan `name` pada setiap `path()` di `urls.py` dianggap sebagai praktik yang sangat baik?